

TP-Film Partie 2

Les relations 1:n

Pour le moment nous n'avons manipulé qu'une table avec Eloquent. Dans le présent chapitre nous allons utiliser deux tables et les mettre en relation.

La relation la plus répandue et la plus simple entre deux tables est celle qui fait correspondre un enregistrement d'une table à plusieurs enregistrements de l'autre table, on parle de relation de un à plusieurs ou encore de relation de type **1:n**.

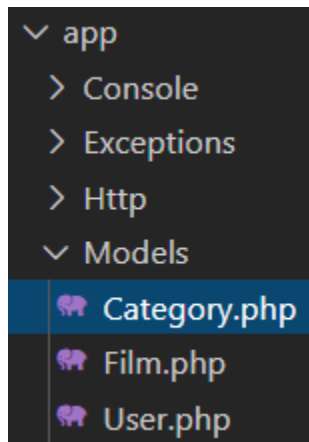
Nous allons poursuivre notre gestion de films. Comme nous en avons beaucoup nous éprouvons la nécessité de les classer en catégories : comédie, fantastique, drame, thriller...

Les migrations

La table categories

Nous allons créer une table pour les catégories. On crée sa migration en même temps que le modèle :

```
php artisan make:model Category --migration
```



Voici le code complété pour la migration :

```
public function up()
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name')->unique();
        $table->string('slug')->unique();
        $table->timestamps();
    });
}
```

On va donc définir les colonnes ;

- **name** : nom de la catégorie
- **slug** : adaptation du nom pour le rendre compatible avec les urls

La table films

On a déjà une migration pour la table **films** mais il va falloir la compléter pour pouvoir la mettre en relation avec la table **categories** :

```
public function up()
{
    Schema::disableForeignKeyConstraints();
    Schema::create('films', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->year('year');
        $table->text('description');
        $table->timestamps();
        $table->softDeletes();
        $table->foreignId('category_id')->constrained()->onUpdate('restrict')->onDelete('restrict');
    });
}
```

Dans la table **films** on déclare une clé étrangère nommée **category_id** qui référence la colonne **id** dans la table **categories**. La méthode **foreignId** crée une colonne de type UNSIGNED BIGINT. La méthode **constrained** utilise les conventions de Laravel pour déterminer le nom de la table référencée, ici c'est **category**.

En cas de suppression (**onDelete**) ou de modification (**onUpdate**) on a une restriction (**restrict**).

Que signifient ces deux dernières conditions ?

Imaginez que vous ayez une catégorie avec l'id 5 qui a deux films, donc dans la table **films** on a deux enregistrements avec **category_id** qui a la valeur 5. Si on supprime la catégorie que va-t-il se passer ? On risque de se retrouver avec nos deux enregistrements dans la table **films** avec une clé étrangère qui ne correspond à aucun enregistrement dans la table **categories**. En mettant **restrict** on empêche la suppression d'une catégorie qui a des films. On doit donc commencer par supprimer ses films avant de le supprimer lui-même. On dit que la base assure l'intégrité référentielle. Elle n'acceptera pas non plus qu'on utilise pour **category_id** une valeur qui n'existe pas dans la table **categories**.

Une autre possibilité est **cascade** à la place de **restrict**. Dans ce cas si vous supprimez une catégorie ça supprimera en cascade les films de cette catégorie.

C'est une option qui est moins utilisée parce qu'elle peut s'avérer dangereuse, surtout dans une base comportant de multiples tables en relation. Mais c'est aussi une stratégie très efficace parce

que c'est le moteur de la base de données qui se charge de gérer les enregistrements en relation, vous n'avez ainsi pas à vous en soucier au niveau du code.

On pourrait aussi ne pas signaler à la base qu'il existe une relation et la gérer seulement dans notre code. Mais c'est encore plus dangereux parce que la moindre erreur de gestion des enregistrements dans votre code risque d'avoir des conséquences importantes dans votre base avec de multiples incohérences.

On va lancer les migrations en rafraichissant la base :

```
php artisan migrate:fresh
```

Les migrations sont effectuées dans l'ordre alphabétique, ce qui peut générer un problème avec les clés étrangères. Si la table référencée est créée après on va tomber sur une erreur du genre :

Illuminate\Database\QueryException : SQLSTATE[HY000]: General error: 1215 Cannot add foreign key constraint (SQL: alter table `films` add constraint `films\_category\_id\_foreign` foreign key (`category\_id`) references `categories` (`id`) on delete restrict on update restrict)

C'est pour cette raison que j'ai ajouté cette ligne dans la migration :

```
Schema::disableForeignKeyConstraints();
```

On désactive temporairement le contrôle référentiel le temps de créer les tables.

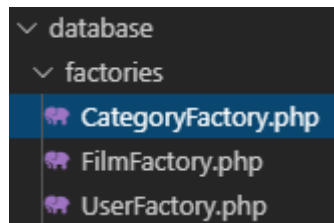
La population

On va remplir les tables avec des enregistrements pour nos essais.

Les catégories

On va définir un certain nombre de catégories. Alors on crée un factory :

```
php artisan make:factory CategoryFactory --model=Category
```



On complète le code :

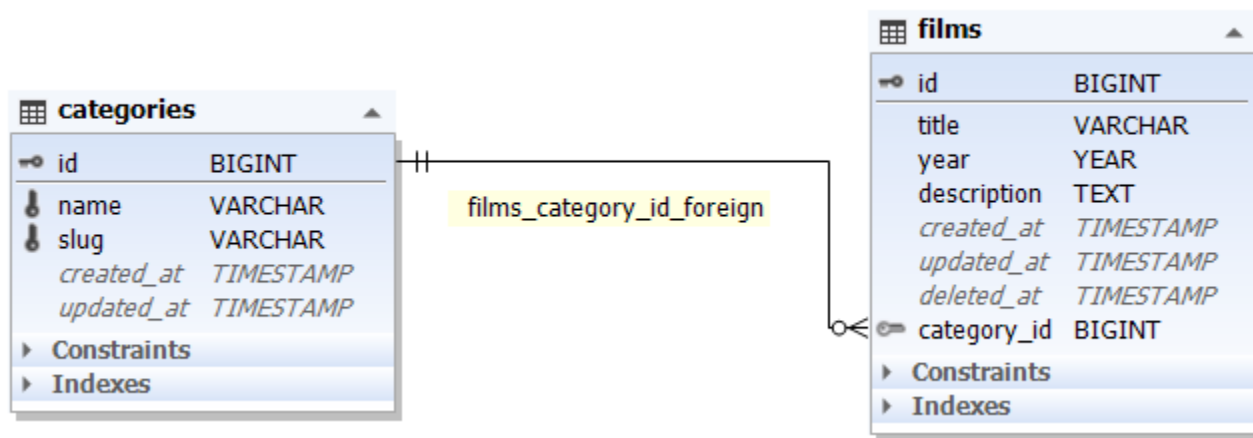
```
public function definition()
{
    $name = $this->faker->word();
    return [
        'name' => $name,
        'slug' => Str::slug($name),
    ];
}
```

La relation

On a la situation suivante :

- une catégorie peut avoir plusieurs films,
- un film n'appartient qu'à une catégorie.

Il faut trouver un moyen pour référencer cette relation dans les tables. Le principe est simple et on l'a vu dans les migrations : on prévoit dans la table **films** une colonne destinée à recevoir l'identifiant de la catégorie. On appelle cette ligne une **clé étrangère** parce qu'on enregistre ici la clé d'une autre table. Voici une représentation visuelle de cette relation :

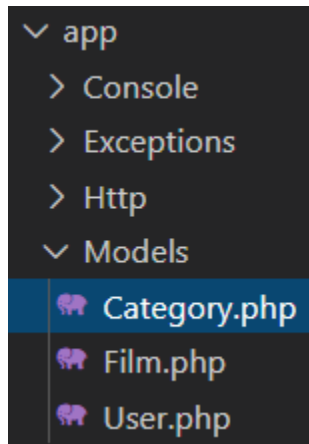


Vous voyez la relation **films_category_id_foreign** dessinée entre la clé **id** dans la table **categories** et la clé étrangère **category_id** dans la table **films**.

Les modèles et la relation

Le modèle Category

Le modèle **Category** se trouve ici (on l'a créé en même temps que la migration) :



On va lui ajouter ce code :

```
public function films()  
{  
    return $this->hasMany(Film::class);  
}
```

On déclare ici avec la méthode **films** (au pluriel) qu'une catégorie a plusieurs (**hasMany**) films (**Film**). On aura ainsi une méthode pratique pour récupérer les films d'une catégorie.

Le modèle Film

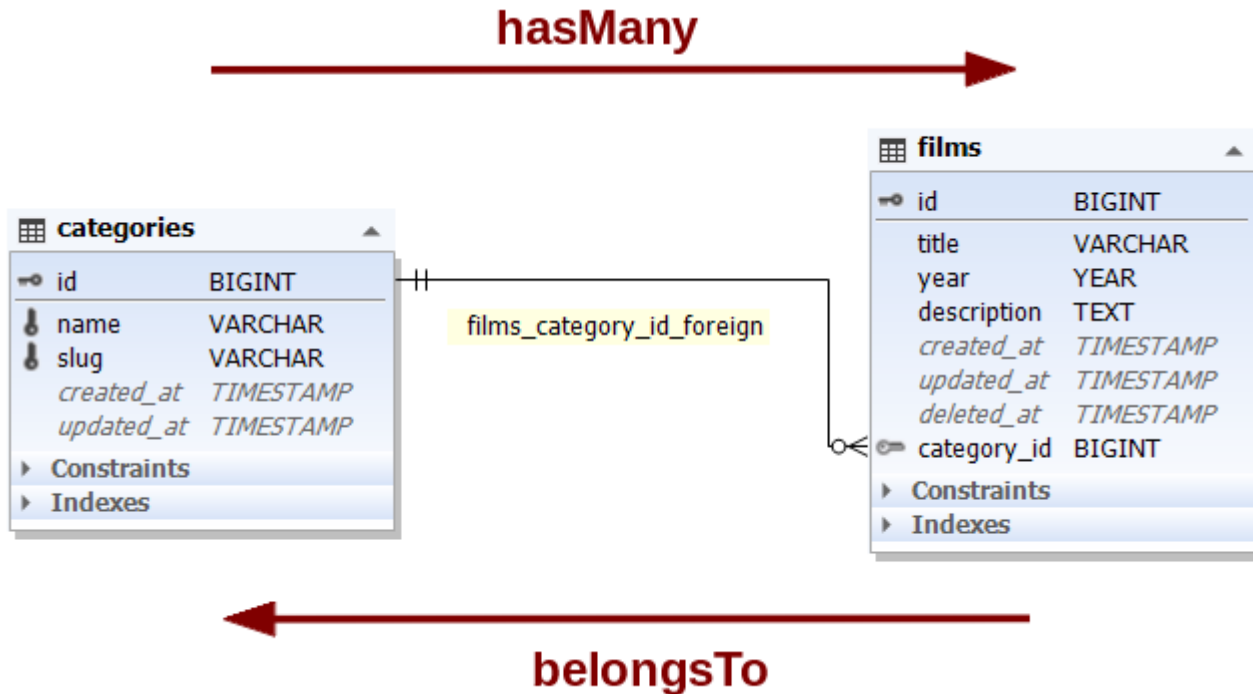
Dans le modèle Film on va coder la réciproque :

```
public function category()  
{  
    return $this->belongsTo(Category::class);  
}
```

Ici on a la méthode **category** (au singulier) qui permet de trouver la catégorie à laquelle appartient (**belongsTo**) le film.

La relation 1:n

Voici une schématisation de la relation avec les deux méthodes :



*Si vous ne spécifiez pas de manière explicite le nom de la table dans un modèle, Laravel le déduit à partir du nom du modèle en le mettant au pluriel (à la mode anglaise) et en mettant la première lettre en minuscule. Donc avec le modèle **Film** il en conclut que la table s'appelle **films**. Si ce n'était pas satisfaisant il faudrait créer une propriété **\$table**.*

Les deux méthodes mises en place permettent de récupérer facilement un enregistrement lié. Par exemple pour avoir tous les films de la catégorie qui a l'id 1 :

```
$films = App\Models\Category::find(1)->films;
```

De la même manière on peut trouver la catégorie du film d'id 1 :

```
$category = App\Models\Film::find(1)->category;
```

Vous voyez que le codage devient limpide avec ces méthodes !

La population (seeding)

Pour la population on va justement utiliser la relation qu'on vient de mettre en place. On modifie ainsi **DatabaseSeeder** :

```
public function run()
{
    // \App\Models\User::factory(10)->create();
    Category::factory()
        ->has(Film::factory()->count(4))
        ->count(10)
        ->create();
}
```

On lance la population :

```
php artisan db:seed
```

Si vous tombez sur une erreur avec duplication de données c'est que faker n'est pas assez aléatoire, alors relancez la migration et la population.

On crée ainsi 10 catégories :

id	name	slug
1	et	et
2	molestias	molestias
3	ipsum	ipsum
4	porro	porro
5	libero	libero
6	quia	quia
7	dignissimos	dignissimos
8	molestiae	molestiae
9	unde	unde
10	sit	sit

Avec cette population pour les catégories le nom et le slug sont indentiques mais dans une situation plus réaliste il y aurait évidemment souvent des différences, avec les majuscules, les espaces, les accents...

Et pour chacune 4 films associés.

id	category_id	title
1	1	Voluptatibus et in.
2	1	Odit maiores.
3	1	Magni cum.
4	1	Eum odit.
5	2	Et et.
6	2	Nam suscipit.
7	2	Qui fugiat quasi.
8	2	Praesentium eos.
9	3	Doloribus ratione.

On a maintenant tout en place pour commencer à nous amuser...

Route et contrôleur

On va définir une nouvelle route pour aller chercher les films par catégorie :

```
Route::controller(FilmController::class)->group(function () {
    Route::delete('films/force/{film}', 'forceDestroy')->name('films.force.destroy');
    Route::put('films/restore/{film}', 'restore')->name('films.restore');
    Route::get('category/{slug}/films', 'index')->name('films.category');
});
```

```
GET|HEAD      category/{slug}/films ..... films.category > FilmController@index
```

On voit qu'on pointe la méthode **index** du contrôleur. Cette méthode existe déjà mais ne servait jusque là qu'à fournir la liste paginée des films sans tenir compte de catégories.

On met à jour le code du contrôleur :

```
public function index($slug = null)
{
    $query = $slug ? Category::whereSlug($slug)->firstOrFail()->films() : Film::query();
    $films = $query->withTrashed()->oldest('title')->paginate(5);
    $categories = Category::all();
    return view('index', compact('films', 'categories', 'slug'));
}
```

On voit qu'on distingue le cas où on fournit un slug de catégorie et le cas où on n'en fournit pas. On envoie toutes les informations nécessaires dans la vue **index**.

La vue index

On va donc un peu modifier la vue **index** pour ajouter cette fonctionnalité. Essentiellement on ajoute une liste de choix :

```
<header class="card-header">
  <p class="card-header-title">Films</p>
  <div class="select">
    <select onchange="window.location.href = this.value">
      <option value="{{ route('films.index') }}" @unless ($slug) selected
@endunless>
        Toutes catégories</option>
      @foreach ($categories as $category)
        <option value="{{ route('films.category', $category->slug) }}"
          {{ $slug == $category->slug ? 'selected' : '' }}>{{ $category-
>name }}</option>
      @endforeach
    </select>
  </div>
  <a class="button is-info" href="{{ route('films.create') }}">Créer un film</a>
</header>
```

On modifie aussi un peu le style pour tenir compte de la liste :

```
select, .is-info {
  margin: 0.3em;
}
```

Maintenant quand on arrive avec l'url **.../films** on a :

Films			
		Toutes catégories ▾	Créer un film
Titre			
Aliquid enim.	Voir	Modifier	Supprimer
Aenean vel elit sed tempus			

Quand on clique sur une option de la liste, donc une des catégories, ça envoie la requête qui va bien et ça affiche les films de cette catégorie :

Films

voluptatum

Cr  er un film

Titre			
Doloremque aspernatur et.	Voir	Modifier	Supprimer
Maxime voluptatum			

Ici on a l'url `.../category/voluptatum/films`.

La vue show

On va aussi ajouter dans la vue **show** le nom de la cat  gorie du film. On compl  te le contr  leur :

```
public function show(Film $film)
{
    $category = $film->category->name;
    return view('show', compact('film', 'category'));
}
```

Et la vue :

```
<p>Ann  e de sortie : {{ $film->year }}</p>
<p>Cat  gorie : {{ $category }}</p>
```

Et   a roule :

Titre : Assumenda autem ea.

Ann  e de sortie : 1992

Cat  gorie : dignissimos

Sapiente itaque quo eius et quidem repellendus laboriosam. Porro sit atque repellat et sint eum impedit. Quam eos odio maiores dolore.

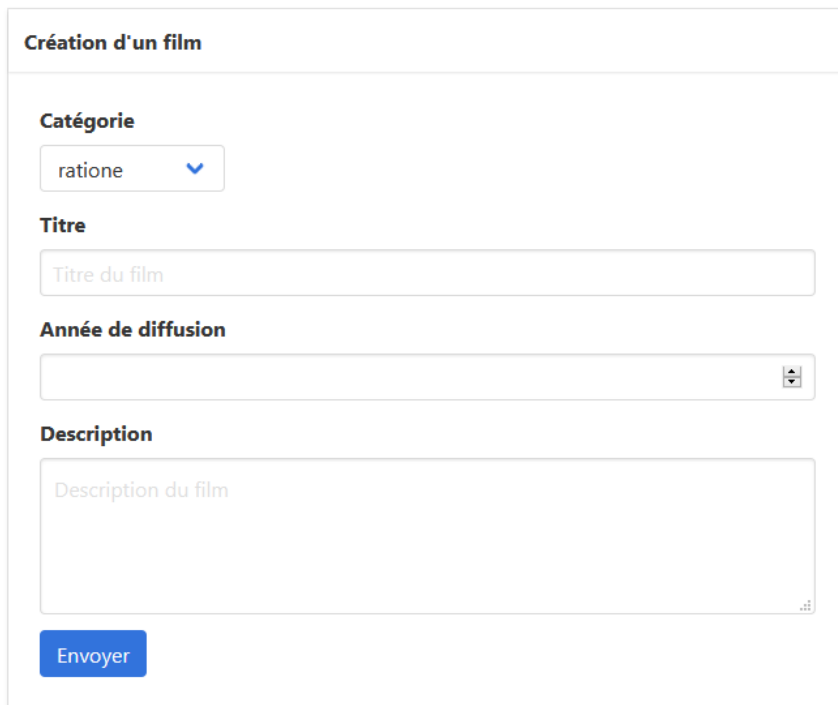
La création d'un film

Maintenant aussi quand on crée un film il faut l'attribuer à une catégorie. Il faut déjà modifier la méthode **create** du contrôleur pour envoyer les catégories :

```
public function create()
{
    $categories = Category::all();
    return view('create', compact('categories'));
}
```

Et modifier la vue **create** en ajoutant une liste des catégories :

```
<div class="field">
<label class="label">Catégorie</label>
<div class="select">
<select name="category_id">
    @foreach ($categories as $category)
        <option value="{{ $category->id }}">{{ $category->name }}</option>
    @endforeach
</select>
</div>
</div>
```



On ajoute le champ **category_id** dans la propriété **\$fillable** du modèle **Film** :

```
protected $fillable = ['title', 'year', 'description', 'category_id'];
```

Et on n'a même pas besoin de toucher à la méthode **store** du contrôleur ! D'autre part on ne va pas se soucier de validation pour une liste imposée.